SECTION 1 | LECTURE 1

이 강의는 무엇을 가르치는가: AI 기능이 아니라 통제 패턴

Spring AI로 배우는 Controlled AI Agent의 출발점.
비용·품질·리스크·흐름을 확실하게 지배하는 프로덕션 레벨 통제 설계 입문.

AXIS 01	AXIS 02	AXIS 03	AXIS 04
COST CONTROL	QUALITY GATE	RISK GUARD	FLOW DESIGN

일반 AI 강의 vs 본 강의의 지향점

TYPICAL AI COURSE

개별 기능 위주의 학습

보통 입문 강의는 API 활용법과 AI의 "기능"을 조각조각 나열하여 학습합니다.

- **Prompt Engineering:** 최적의 답변을 얻기 위한 템플릿 작성법
- **Model Selection:** 다양한 LLM 및 임베딩 모델 연동법
- **Simple RAG / Tool:** 데이터 로드 및 로컬 툴 선언 방식

⚠ 문제점: 기능을 조각으로 알면 운영 시스템 구축이 불가능합니다.

THIS COURSE

운영 및 통제 아키텍처

기능 너머에 존재하는 시스템 안전장치와 통제 패턴을 통합적으로 학습합니다.

- **Cost & Quality Gate:** 토큰 소모 모니터링 및 실시간 평가 루프
- **Risk Guardrail:** 에이전트의 위험 액션 실행 차단 및 검증
- **System Observability:** 프로덕션 운영을 위한 정밀 분산 추적(Trace)

"기능은 단순한 도구일 뿐, 핵심은 아키텍처 수준의 통제 설계입니다."

에이전트 설계의 핵심 통제 4축 (Control Axes)

AXIS 01

비용 (Cost)

무한 루프 방지 및 리소스 제어

- 불필요한 반복 호출 제한
- 토큰 사용량 실시간 관제
- 경량 로컬 모델 최적 조합

AXIS 02

품질 (Quality)

일관된 도메인 스키마와 응답 신뢰성

- LLM 응답 스키마 정합성 보장
- 구조화된 정밀 평가기 연동
- RAG 품질 제어 필터링

AXIS 03

리스크 (Risk)

악성 동작 방지 및 가드레일 수립

- 잘못된 API 액션 파괴 행위 보호
- 환각(Hallucination) 필터링
- 개인정보 노출 원천 차단

AXIS 04

흐름 (Flow)

순차/병렬 오케스트레이션 설계

- 구조적 상태 전이 관리
- 명확한 에이전트간 협력 규칙
- 개발자 추적 가능한 가시성

Spring AI 생태계를 통제 패턴으로 조망하기

단순히 API 사용법을 아는 것에서 벗어나, 각 요소가 전체 운영 아키텍처의 **어디에 위치하는지** 이해해야 합니다.

01. ENTRY POINT	`ChatClient`	→ 시스템으로 들어오는 요청 경로와 기본적인 프롬프트 컨텍스트를 설계합니다.
02. INJECTION GATE	`RAG (Vector Store)` `Tool Call (Function Callback)` `Advisor System`	외부 리소스와 격리 조치, 안전한 비즈니스 정책을 주입합니다.
03. OBSERVABILITY	`Trace Metric & Evaluator`	→ 실행 과정을 정밀 모니터링하여 품질 지표를 측정하고 피드백 루프를 완성합니다.
04. ORCHESTRATION	`Agentic Patterns`	→ 라우팅 및 협업 전략을 세워 통제 하에 복합 워크플로우를 처리하도록 구성합니다.

실무 예제 분석: Incident Responder 도메인

왜 이 예제 도메인이 필요한가?

우리가 단순한 Hello World형 챗봇이 아니라, 높은 신뢰도를 요구하는 시스템 장애 자동 대응 도메인을 채택한 명확한 아키텍처 관점의 이유가 존재합니다.

- **강력한 리스크 노출:** 프로덕션 장애 발생 시, 잘못 유추된 가이드나 툴 호출은 전체 서버의 즉각적인 파멸적 피해를 초래합니다.
- **실시간 정보 연동의 필수성:** 시스템 메트릭, 최신 기술 매뉴얼 등의 정확한 맥락 주입 품질 가이드라인이 필요합니다.

◎실서비스에서 마주할 가장 선명한 통제 시나리오를 대변합니다.

종합 아키텍처 연동 증명

단일 시나리오 안에서 프레임워크가 요구하는 모든 복합 패턴을 일관적이고 아름답게 설계해 나갑니다.

- **RAG (검색 증강):** 장애 관련 런북 가이드를 시스템에 빠르고 안전하게 공급
- **Tool (도구 위임):** 현재 파드 상태 조회, 리포트 발행 자동화 실행
- **Observability (추적성):** 실행 트레이스를 기반으로 소요 토큰량 추적
- **Evaluator (신뢰 검증):** 대응 브리프 정보의 정합성을 AI 기반으로 평가

"통제 4축" 을 완벽히 보여줄 가장 선명한 플레이그라운드입니다.

통제 아키텍처의 단계적 성장: Part 1 & 2 로드맵

PART 1

Spring AI 기반 통제 기초

프레임워크의 표준 아티팩트들을 통제 아키텍처 관점으로 전환시켜 시스템에 안착시키는 핵심 토대를 닦습니다.

- 기능을 **통제 포인트 (Control Points)**로 재조정하여 결합
- Observability 도구를 사용해 시스템 내부 추적 가시성 획득
- 비용 모니터링 및 실시간 LLM 품질 평가 게이트 구축

"기본 통제 체계와 운영 감각 확립"

PART 2

Embabel을 통한 구조적 설계

단순 툴 체이닝을 넘어, 고차원의 인지 프레임워크(Cognitive Engine)를 도입해 에이전트를 선언적 지배 구조로 진화시킵니다.

- **Goal / Plan / Action** 개념 기반의 정교한 동적 실행 흐름
- Typed Domain Model을 적용하여 구조화 데이터 엄격 통제
- 고수준 추상화를 바탕으로 복합 상황의 지능형 제어 수행

"안정적인 지배 구조(Governance) 설계"

오늘 정리 & 다음 수업 안내

TODAY KEYTAKEAWAYS

핵심 관점 요약

- **AI 기능 아키텍처의 대전환:** 단순히 "할 수 있는가"가 아니라 안정적으로 "통제 가능한가"가 실무 아키텍처의 중심 가치입니다.
- **공통 설계 지표, 통제 4축:** 전체 과정을 흐르는 4대 주춧돌인 비용, 품질, 리스크, 흐름을 항상 의식하고 코드를 설계해야 합니다.
- **Spring AI에서 Embabel로:** 기초 통제 체인(Part 1)을 다진 후, 더 지능적인 아키텍처 구조화(Part 2)로 이어집니다.

#spring_ai #controlled_agent #orientation

UP NEXT LECTURE

다음 수업 미리보기

본격적인 환경 구성에 들어가기에 앞서, 실무에서 마주치는 현실 세계의 참혹한 장벽과 비즈니스 실패 사례들을 가감 없이 분해해봅니다.

LECTURE 2 TOPIC

왜 통제가 필요한가: 비용·품질·리스크·흐름 4축 문제의식 실전 딥다이브

Ready to Start